Trajectory Transformer: Reinforcement Learning as One Big Sequence Modeling Problem

백승언

13 Aug, 2023

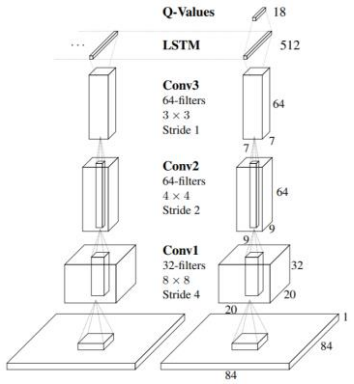
Contents

Introduction

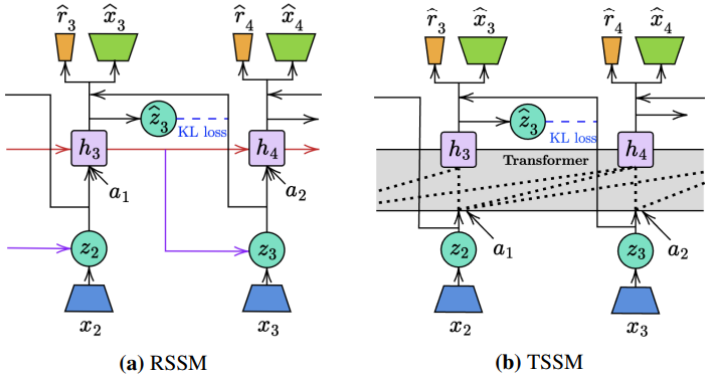
Sequential Neural Network in Reinforcement Learning

- **Various models have utilized the sequential neural networks such as LSTMs, Seq2Seq models and Transformer architectures**
  - Policy: ALD(Transformer), …
  - Value: DRQN(LSTM), FRMQN(memory network), …
  - Transition model: Dreamer(LSTM), TransDreamer(Transformer), …
  - Multi-agent RL: QMIX(GRU), AlphaStar(LSTM), …

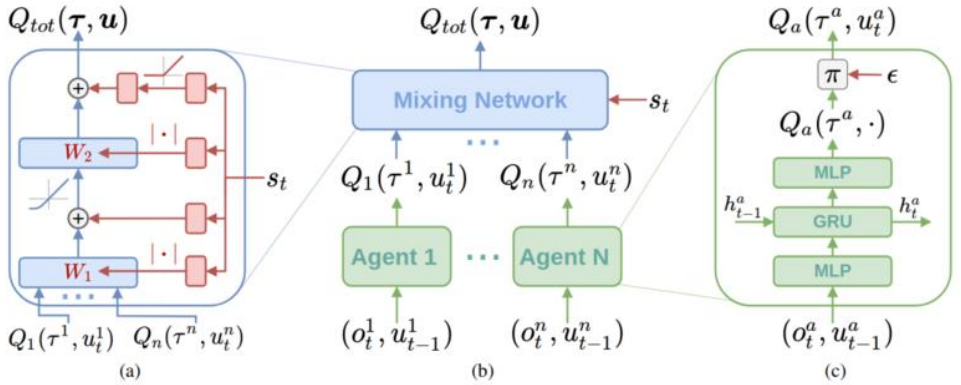
- **While, previous works demonstrated the importance of such models for representing memory, they still rely on standard RL algorithmic advances to improve performance.**
  - The goal in **trajectory transformer** is different: They aim to **replace** as much of the RL pipeline as possible with sequence modeling



DRQN architecture



Draemer / TransDreamer architecture



QMIX architecture

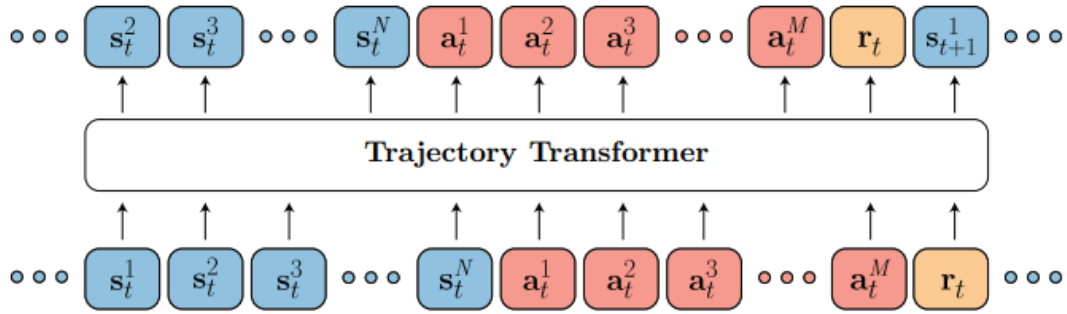
Trajectory Transformer

Overview

- **Overview of the Trajectory Transformer**
  - Previous model-free, model-based and offline RL algorithms require the following components
    - Model-free algorithms: critic, actor(optional)
    - Model-based: dynamics model(optional), critic(optional), actor(optional)
    - Offline RL algorithms: dynamics model(optional), critic(optional), behavior constraints
  - However, proposed Transformer-based model unified all components under single sequence model
    - The advantages of this perspective is that high-capacity sequence model architectures can be brought to resolve the problem, resulting in an approach that could benefit from the scalability underlying large-scale learning results
  - Additionally, proposed model could achieve various tasks including imitation learning, goal-reaching, offline RL tasks with simple modification on the same decoding procedure
    - Their results suggest that the algorithms and architectural motifs that have been widely applicable in unsupervised learning carry similar benefits in reinforcement learning
  - They proposed a Transformer-based model for predicting the observation, action, and reward, and optimized the objective function as follows
    - $\mathcal{L}(\bar{\tau}) = \Sigma_{t=0}^{T-1}\left(\Sigma_{i=0}^{N-1}\log P_\theta\left(\bar{\mathbf{s}}_t^i \mid \bar{\mathbf{s}}_t^{<i}, \bar{\tau}_{<t}\right) + \Sigma_{j=0}^{M-1}\log P_\theta\left(\bar{\mathbf{a}}_t^j \mid \bar{\mathbf{a}}_t^{<j}, \bar{\tau}_{<t}\right) + \log P_\theta\left(\bar{r}_t \mid \bar{\mathbf{a}}, \bar{\mathbf{s}}, \bar{\tau}_{<t}\right)\right)$
  - Evaluation demonstrated that Trajectory Transformer showed significant performance in imitation learning tasks, goal-reaching tasks, and offline RL tasks

Model specification (I)

- **Trajectory data as unstructured sequence for modeling by a Transformer architecture**
  - A trajectory τ consists of N-dimensional states, M-dimensional actions, and scalar rewards
    - $\tau = \{s_t^0, s_t^1, \ldots, s_t^{N-1}, a_t^0, a_t^1, \ldots, a_t^{M-1}, r_t\}_{t=0}^{T-1}$, i *denotes dimension*, t *denotes timestep*
  - Discretizing inputs(tokenization)
    - They investigated two simple discretization approaches
      - **Uniform**: Assuming a per-dimension vocabulary size of V, the tokens for state dimension i cover uniformly-spaced intervals of width $(\max \mathbf{s}^i - \min \mathbf{s}^i)/V$
      - **Quantile**: All tokens for a given dimension account for an equal amount of probability mass under the empirical data distribution; each token accounts for 1 out of every V data points in the training set.
  - Using Transformer decoder mirroring the GPT architecture
    - They used small-scale language model consisting of four layers and six self-attention heads

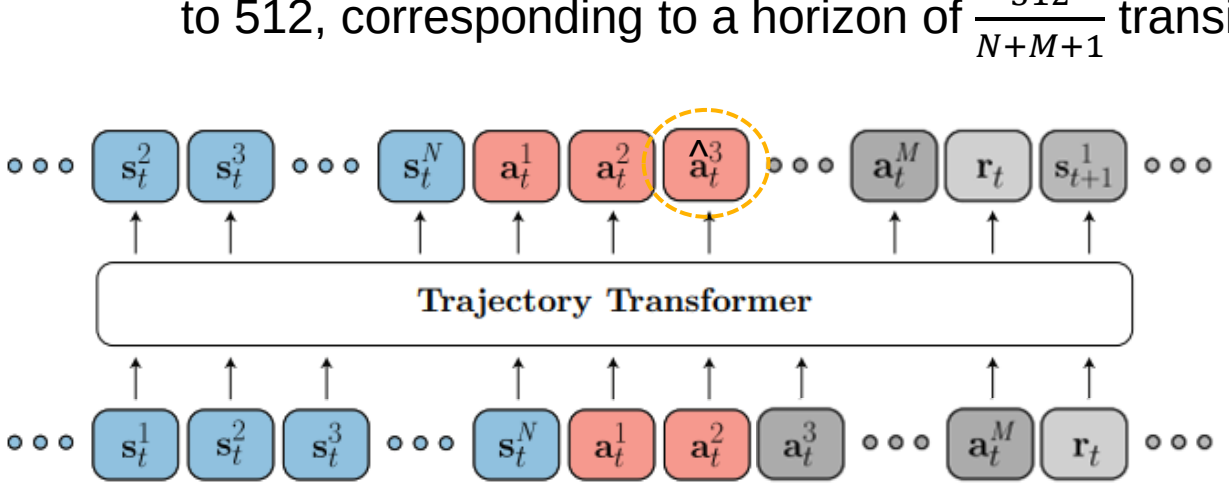


Architecture of the Trajectory Transformer

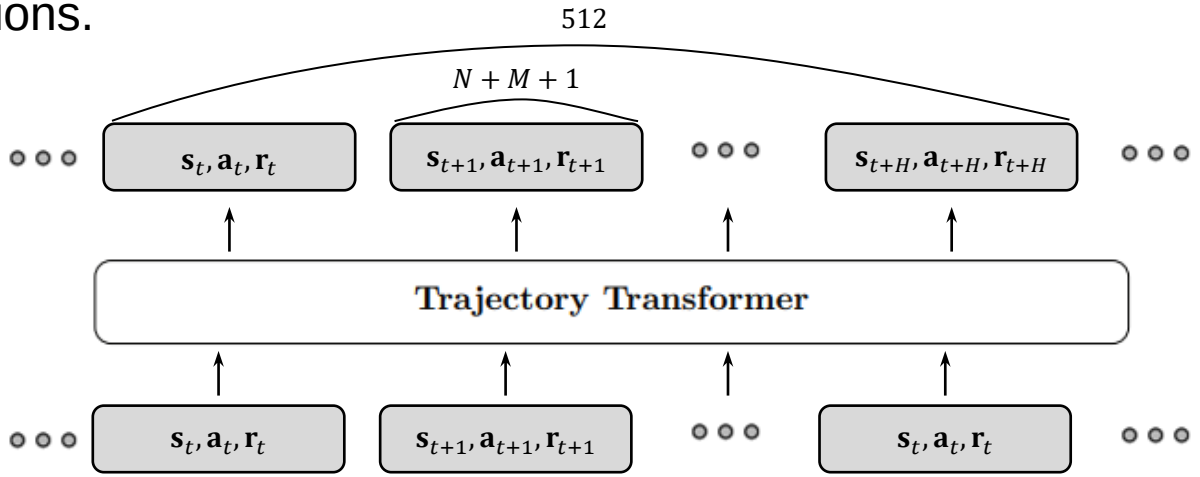
Model specification (II)

- **Objective function**
  - The authors optimized the following objective, which is the standard teacher-forcing procedure used to train autoregressive recurrent models.
    - $\mathcal{L}(\bar{\tau}) = \Sigma_{t=0}^{T-1}\left(\Sigma_{i=0}^{N-1}\log P_\theta\left(\bar{\mathbf{s}}_t^i \mid \bar{\mathbf{s}}_t^{<i}, \bar{\tau}_{<t}\right) + \Sigma_{j=0}^{M-1}\log P_\theta\left(\bar{\mathbf{a}}_t^j \mid \bar{\mathbf{a}}_t^{<j}, \bar{\tau}_{<t}\right) + \log P_\theta(\bar{r}_t \mid \bar{\mathbf{a}}, \bar{\mathbf{s}}, \bar{\tau}_{<t})\right)$,
    - in which they use $\bar{\tau}_{<t}$ as a shorthand for a tokenized trajectory from timesteps 0 through t-1

- **Prediction horizon**
  - Due to the quadratic complexity of self-attention, they limited the maximum number of conditioning tokens to 512, corresponding to a horizon of $\frac{512}{N+M+1}$ transitions.



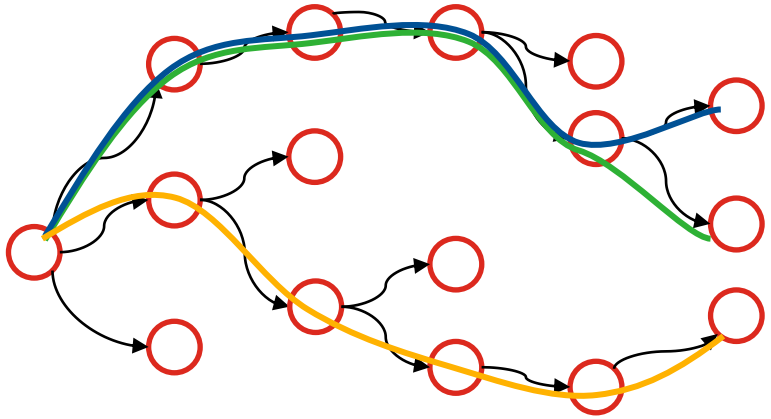
Casual transformer



Horizon of Trajectory Transformer

Planning technique based on tasks (I)

- **Beam search(BS)**
  - Heuristic technique for generating sequence widely used in NLP task
    - BS selects the K-sequences which has high log-probability at each step



Example of Beam search(K=3)

- **Planning according to tasks**
  - For imitation learning
    - This situation matches the goal of sequence modeling exactly.
      - They used BS without modification by setting the conditioning input $\mathbf{x}$ the current state s_t
  - For goal-conditioned RL
    - Proposed Transformer architecture features a “causal” attention mask to ensure that predictions only depends on previous tokens in sequence.
    - However, for goal-conditioned RL, they conditioned the goal state or final state $\mathbf{s}_T$
      - They decode trajectories with probabilities of the form: $P_\theta(s_t^i \mid \mathbf{s}_t^{<i}, \boldsymbol{\tau}_{<t}, \mathbf{s}_T)$

Planning technique based on tasks (II)

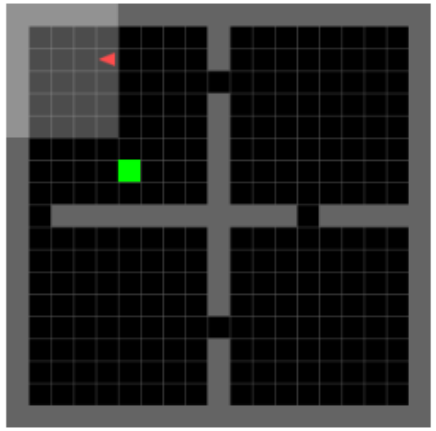
- **Return-to-go(reward-to-go)**
  - Sum of reward along the trajectory until that time t
    - $R_t = \Sigma_{t'=t}^{T} \gamma^{t'-t} r_t$

- **Planning according to tasks**
  - For offline RL
    - By replacing the log-probabilities of transitions with the predicted reward Signal, they could utilize the same Trajectory Transformer and search strategy for reward maximizing behavior.
    - However, using beam search as a reward-maximizing procedure has the risk of leading to myopic behavior
      - To address this issue, they augment each transition in the training trajectories with return-to-go $R_t = \Sigma_{t'=t}^{T} \gamma^{t'-t} r_t$ and include it as an additional quantity, discretized identically to the others.
    - Specifically, the model sample full transitions $(\mathbf{s}_t, \mathbf{a}_t, r_t, R_t)$ using likelihood-maximizing beam search, treat these transitions as our vocabulary, and filter sequences of transitions by those with the highest cumulative reward plus return-to-go estimate

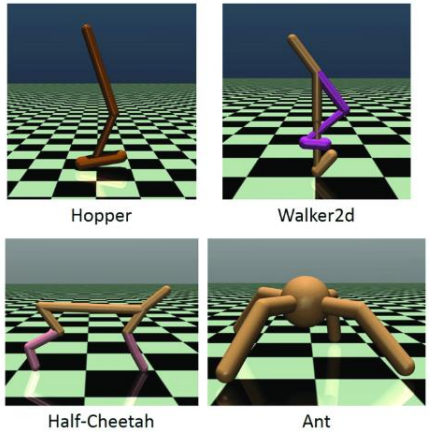
Experiments

Environment and Dataset

- **Environment**
  - Four room environment
    - Environment that agent must navigate in maze composed of four rooms
    - To obtain a reward, the agent must reach the goal square. Both the agent and the goal square are randomly placed in any of the four rooms.
  - MuJoCo environment
    - Environment includes diverse visual control tasks
      - Halfcheetah, Humanoid, Walker, and so on



Four room env

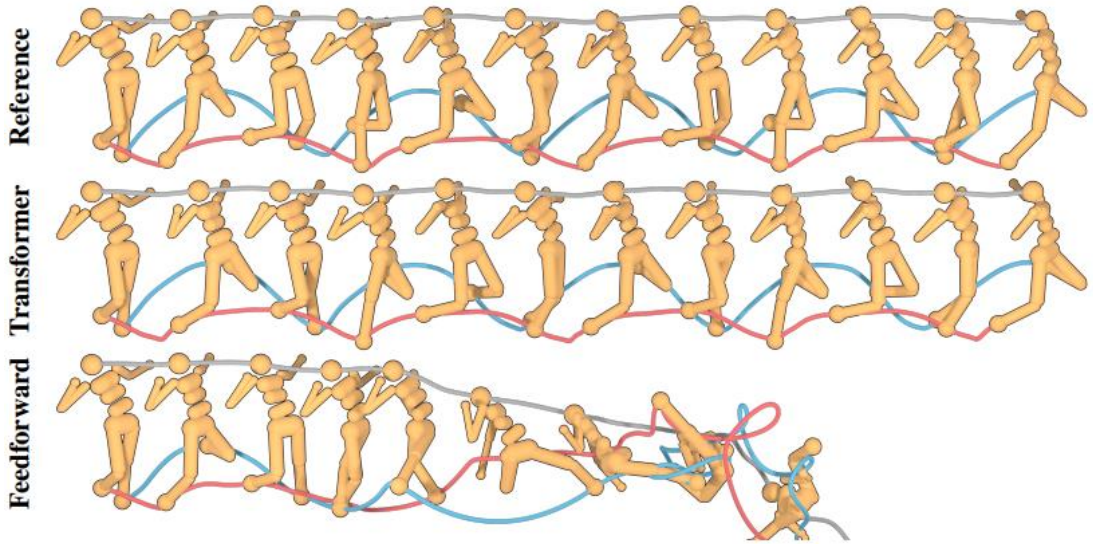


MuJoCo Env

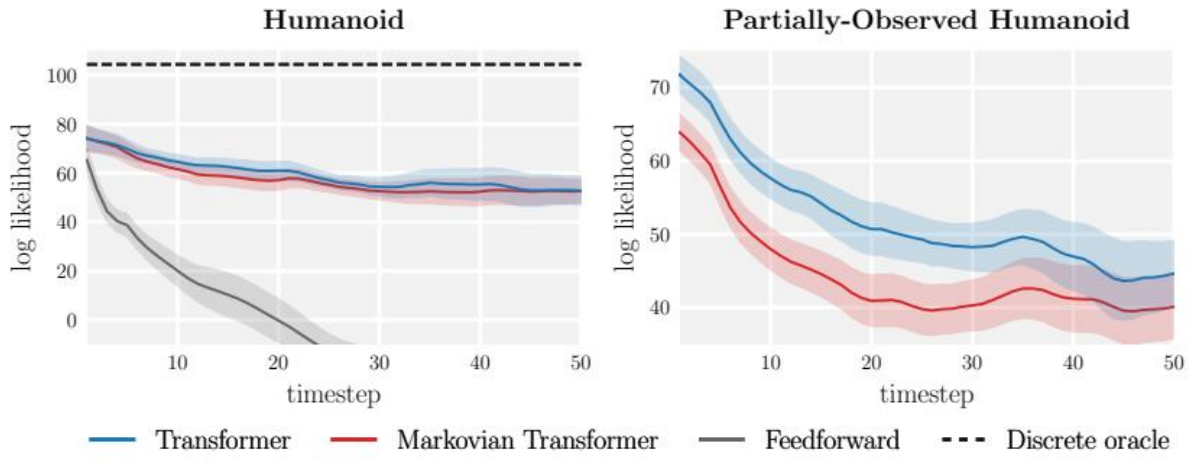
- **D4RL: Datasets for Deep Data-Driven RL**
  - D4RL is data collection of well-known envs for offline RL
    - Maze2D, AntMaze, MuJoCo, and so on
  - Expert levels in D4RL MuJoCo
    - **Medium**: generated by first training a policy online using SAC (1M)
    - **Medium-replay(Mixed)**: consist of recoding all samples in the replay buffer observed during training until policy reaches the medium level performance
    - **Med-expert**: mixing equal amounts of expert demonstrations and suboptimal data, generated via a partially trained policy or by unrolling a uniform-at-random policy

Experimental results (I)

- **Model prediction results**
  - Experiments show better trajectory prediction performance compared to previous SOTA planning model
    - The trajectory(100-step) generated by proposed model shows visually indistinguishable from those original dataset, while in the single-step model, compounding errors lead to unsuitable trajectory predictions
  - The authors also compared proposed model(causal transformer) and Markovian transformer(1-step) model
    - In fully observable setting and partially observable setting(50% of states were randomly masked)
      - Proposed model demonstrated the marginally superior accuracy in partial observable setting compared than Markovian Transformer



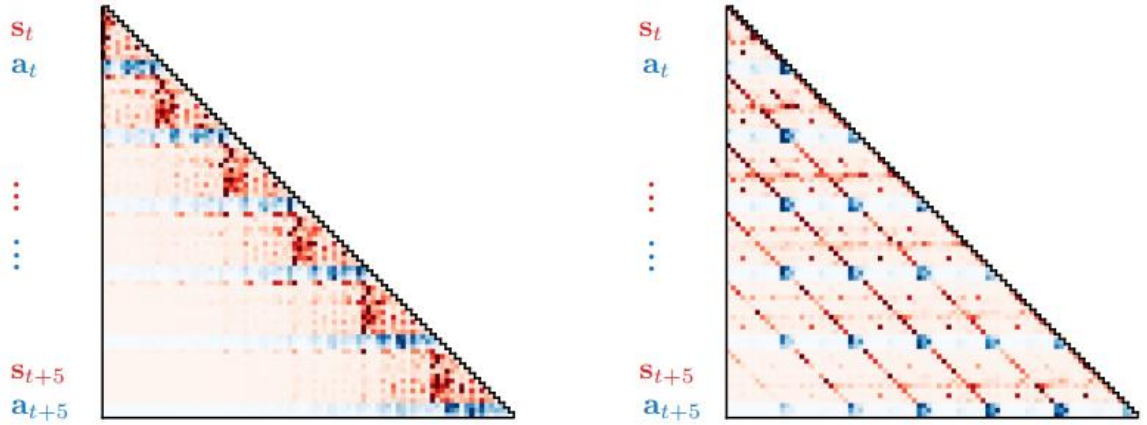
Generated trajectory in Humanoid



Accuracies of generated trajectory in Humanoid

Experimental results (II)

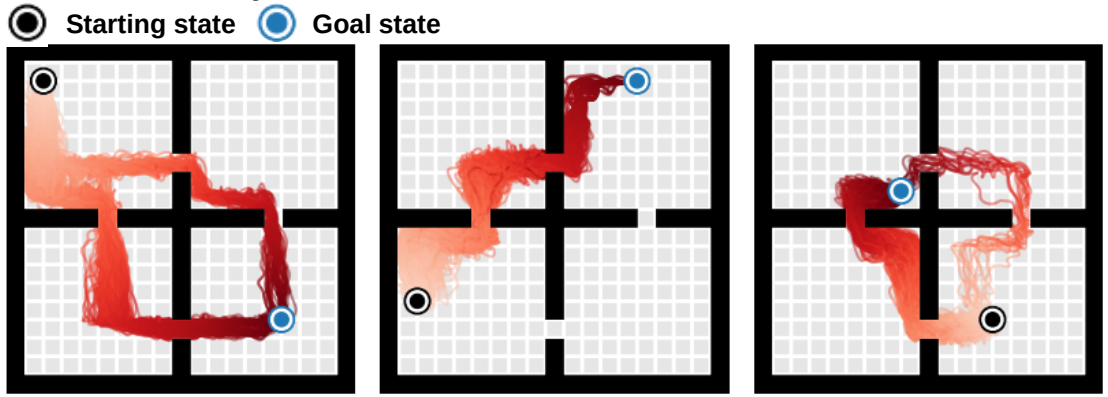
- **Analysis of attention pattern**
  - The authors reported two distinct attention patterns during trajectory prediction(in Hopper)
    - Left: Both states and actions are dependent primarily on the immediately preceding transition
      - Markov property
    - Right: Surprisingly, actions rely more on past actions than they do on past states



Attention pattern(first and third layer attention head) in Hopper

- **Results in Imitation learning task**
  - Proposed model achieves an average normalized return
    - Return of 104% and 109% in the Hopper and Walker 2d env, respectively

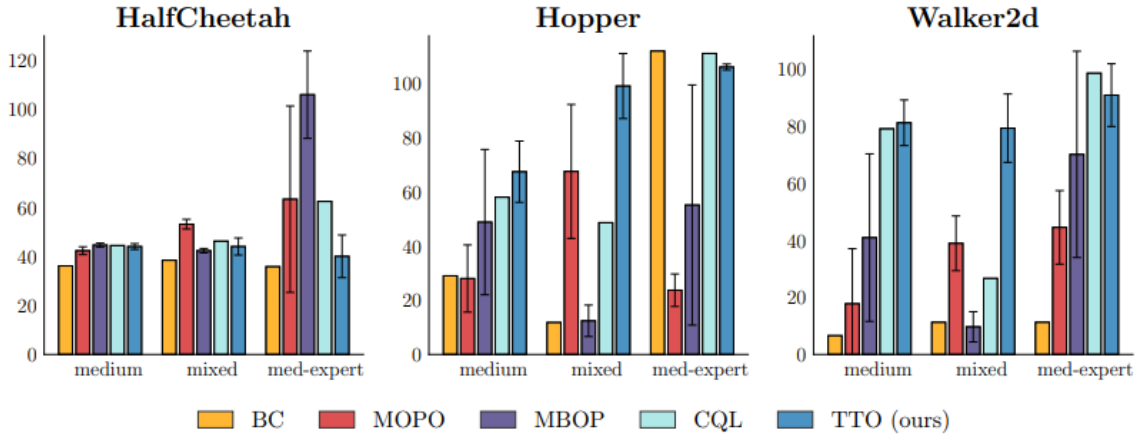
- **Results in goal-reaching task**
  - Proposed model accomplished the goal-reaching task with no reward shaping, reward
    - The below figures show that generated trajectories in four room env



Trajectories of goal-reaching task collected by TTO

Experimental results (III)

- **Comparison with previous methods in Offline RL tasks**
  - Experiments showed comparable performance compared to previous SOTA methods in D4RL offline RL benchmark(HalfCheetah, Hopper, Walker2d)
    - CQL (Model-free RL)
    - MOPO (Model-based RL)
    - MBOP (Model-based planning)
    - BC (Behavior cloning)
    - TTO(Proposed model)
  - The authors assumed that the reason for the low performance in the HalfCheetah and med-expert dataset setting was that the discretization of return was not refined as the performance of the expert data rapidly improved.



Offline RL results

Environment	Dataset type	BC	TTO (ours)	CQL	MOPO	MBOP
halfcheetah	medium	36.1	44.0 ± 1.2	44.4	42.3 ± 1.6	44.6 ± 0.8
halfcheetah	mixed	38.4	44.1 ± 3.5	46.2	53.1 ± 2.0	42.3 ± 0.9
halfcheetah	med-expert	35.8	40.8 ± 8.7	62.4	63.3 ± 38.0	105.9 ± 17.8
hopper	medium	29.0	67.4 ± 11.3	58.0	28.0 ± 12.4	48.8 ± 26.8
hopper	mixed	11.8	99.4 ± 12.6	48.6	67.5 ± 24.7	12.4 ± 5.8
hopper	med-expert	111.9	106 ± 1.1	111.0	23.7 ± 6.0	55.1 ± 44.3
walker2d	medium	6.6	81.3 ± 8.0	79.2	17.8 ± 19.3	41.0 ± 29.4
walker2d	mixed	11.3	79.4 ± 12.8	26.7	39.0 ± 9.6	9.7 ± 5.3
walker2d	med-expert	11.3	91.0 ± 10.8	98.7	44.6 ± 12.9	70.2 ± 36.2

Offline RL results in tabular form

Thank you!

Q&A